

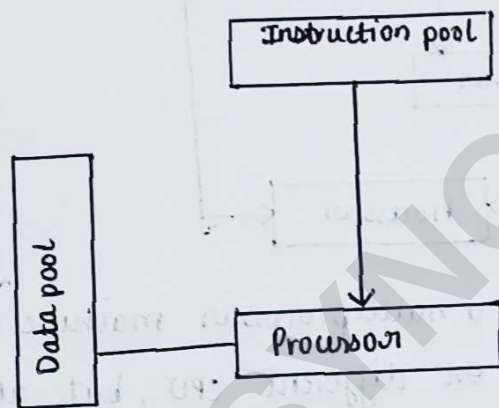
Parallel Computing

Module 1

① classify parallel computer and explain.

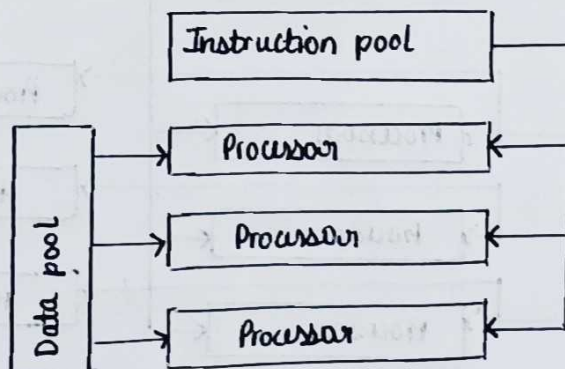
→
1) Parallel computers are typically classified using Flynn's taxonomy: SISD, SIMD, MISD and MIMD.

2) SISD systems (Single instruction, single data system):



- * An SISD computing system is a uniprocessor machine which is capable of executing a single instruction, operating on a single data stream.
- * In SISD, machine instructions are processed in a sequential manner and computers adopting this model are popularly called sequential computers.

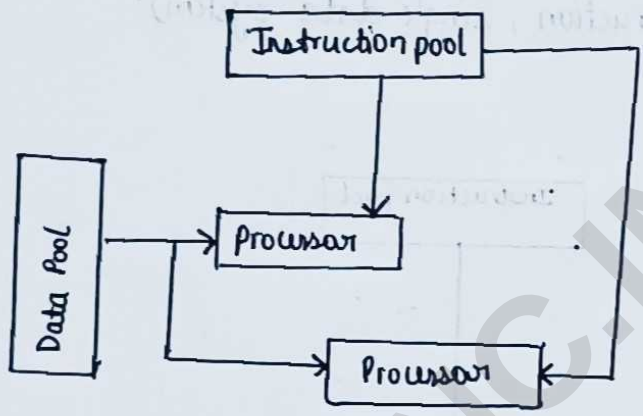
3) Single instruction, multiple data systems (SIMD):



~~different~~ ~~SIMD~~ computing system is a multiprocessor machine capable of executing same instruction on all the CPU, but operating on different data streams.

- * SIMD is efficient for vector operations, such as in graphics and scientific computing.

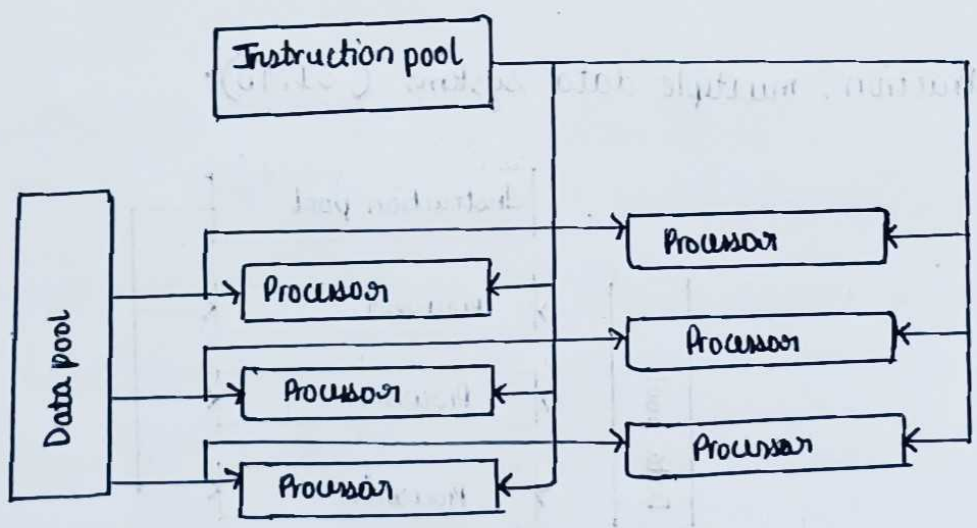
3) Multiple instruction, single data systems (MISD)



- * An MISD computing system is a multiprocessor machine capable of executing different instruction on different CPU, but all of them operating on the same dataset.

- * Example $Z = \sin(x) + \cos(x) + \tan(x)$, the system performs different operations on the same data set.

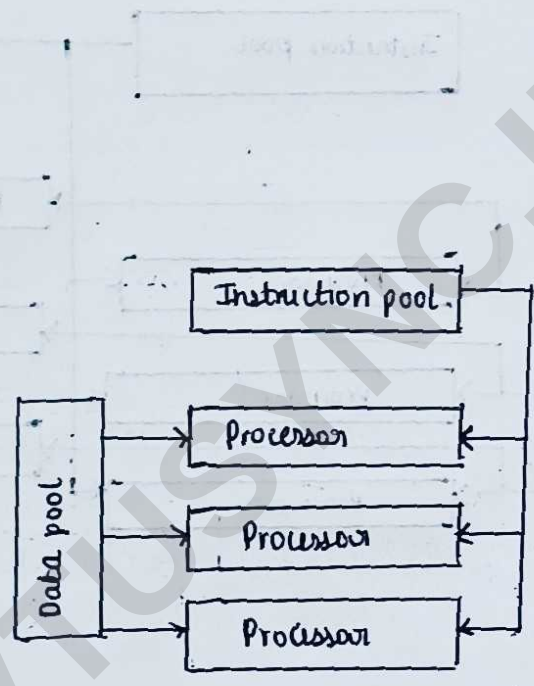
4) Multiple instruction, multiple data systems (MIMD):



- * An MIMD system is a multiprocessor machine which is capable of executing multiple instruction on multiple data sets.
- * Each CPU in the MIMD model has a separate instruction and data streams, therefore machine built using the model are capable of any kind of application. Unlike SIMD and ~~MIMD~~ MISD machines, CPU's in MIMD machines work asynchronously.

② Explain SIMD systems and MIMD systems.

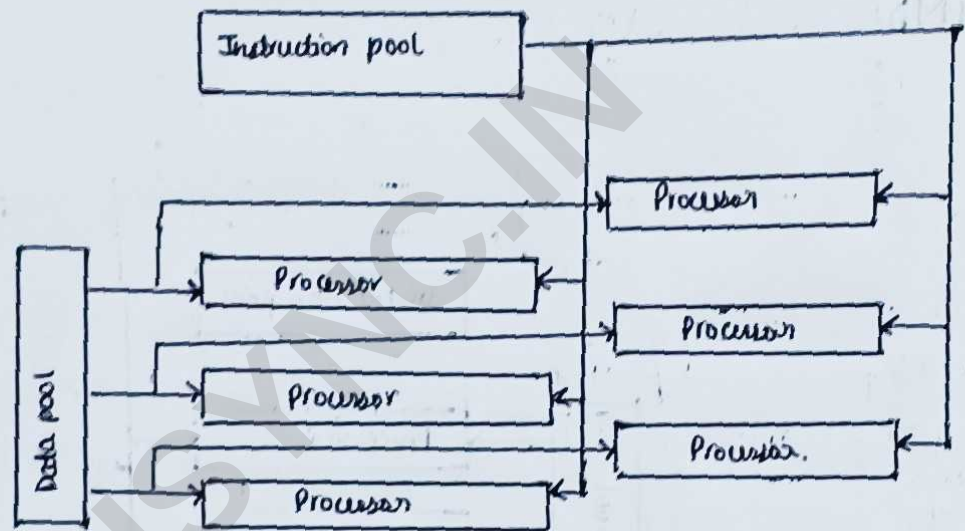
→ SIMD:



- 1) An SIMD system is a multiprocessor machine capable of executing the same instruction on all CPU but operating on different data streams.
- 2) Machine based on SIMD model are well suited to scientific computing since they involve lots of vector and matrix operations.
- 3) so that the information can be passed to all the processing elements organized data elements of vectors can be divided into multiple sets (N sets for N, PE systems) and each PE can process one data set.
- 4) Each processing element typically has its own local memory
- 5) Ideal for data-parallel problems, like image processing, matrix operations

- and simulations.
- c) High performance on tasks that performs the same operation on large dataset.
 - d) Hardware is relatively simple and power efficient.
 - e) Example: Adding two vectors of size 1000 where each element is processed in parallel using the same addition instruction.

* MIMD systems:



- 1) An MIMD system is a multiprocessor machine which is capable of executing multiple instructions on multiple datasets.
- 2) Each processing element in the MIMD model has separate instruction and data stream, therefore machines built using this model are capable to any kind of application. Unlike SIMD and ~~MIMD~~ MISD machines processing elements in MIMD work asynchronously.
- 3) Processors may or may not share memory, classified into shared memory and distributed memory MIMD.
- 4) In shared memory MIMD, all the processing elements are connected to a single global memory and they all have access to it.

- 5) In distributed MIMD, all processing elements have a local memory
- 6) MIMD systems are more flexible than SIMD and can support a wider range of applications.
- 7) Communication can be through shared memory or message passing (MPI)
- 8) Example: One processor running a web server while another runs a database engine. - both doing completely different tasks.

③ Explain interconnection networks in parallel systems?

→ 1) shared memory interconnection networks:

a) Bus-Based interconnect

- 1) All processors share a common communication bus.
- 2) Becomes a bottleneck when many processors try to communicate at once

b) Crossbar switch:

- 1) Provides dedicated pathways between each processors and each memory module.
- 2) High speed and parallel communication possible.
- 3) Expensive due to the large number of switches required

c) Multistage interconnection network:

- 1) Uses multiple layers of switches to connect processors to memory modules.
- 2) Less expensive than crossbar, more scalable than buses.

d) Multicore shared memory:

- 1) Modern CPUs with multiple cores sharing L2/L3 cache and main memory
- 2) Interconnection via a high speed internal bus or ring.

2) Distributed memory interconnection networks:

a) Ring

- 1) Each processor connects to two neighbours
- 2) Latency increases with distances between processors.
- 3) Simple and low cost

b) 2D Mesh / Grid

- 1) Processors arranged in a 2D grid, each connected to its 4 neighbours.
- 2) Scales better than ring.
- 3) Can suffer from congestion near the center.

c) Torus

- 1) Like mesh, but with wrap-around connections (edges connect back).
- 2) Improves latency and bandwidth compared to a plain mesh.

d) Hypercube:

- 1) Logically connects processors in a binary n -dimensional cube
- 2) Fast communication with low diameter
- 3) Complex wiring as dimensionality increases.

e) Fully connected:

- a) Every processor connects to every other processor
- b) Offers minimal latency but is not scalable.

④ How are processes and threads coordinated in parallel programming.

→

- 1) Coordination ensures correct data access and task synchronization.
- 2) Mutual exclusion (mutexes) prevents simultaneous access to critical sections.
- 3) Semaphores help manage access to resources.
- 4) Barriers synchronize all threads at specific points.
- 5) Condition variables enable threads to wait for specific conditions.
- 6) Message passing is used in distributed systems to coordinate processes.
- 7) Incorrect coordination can lead to race condition or deadlocks.
- 8) Coordination overhead impacts parallel program performance.

⑤ Explain about shared memory in detail.

→ Shared memory systems are parallel computer systems in which all processors share a single address space, and can access all parts of memory directly.

- 1) Single address space: All threads / processors refer to memory using the same address.
- 2) Ease of programming: Threads can naturally share data using regular variables.
- 3) Thread-Based Parallelism: often used with Pthreads or OpenMP to parallelize task using threads.
- 4) Data sharing: Multiple threads can access and modify shared variables, which make synchronization important.

- 5) Efficient communication: since data is directly accessible, communication between threads is faster than in distributed system.
- 6) Synchronization Required: To avoid race conditions, you must use mechanisms like mutexes, semaphores, or barriers.
- 7) Cache Coherence Issues: when multiple cores cache shared data, maintaining consistency becomes a challenge.
- 8) Limited scalability: works well for small to medium number of processors. Performance drops as the number of cores grow due to memory contention.
- ⑥ Explain about distributed memory in detail.

→ A distributed memory system is a system where each processor has its own local memory and processors communicate by explicitly sending and receiving messages.

- 1) Private memory per processor: Each processor has exclusive access to its memory, inaccessible to others.
- 2) Communication via message passing: To share data, processors must use a message passing interface, like MPI.
- 3) No shared address space: Variables in one processor's memory cannot be directly read or modified by another processor.
- 4) Highly scalable: Suitable for large scale systems, such as clusters or supercomputers.
- 5) Explicit data movement: Programmers must determine what data to send / receive and when to do it.
- 6) Network connection Required: Nodes are connected by network like ethernet, InfiniBand or custom interconnects.

- 7) Lower memory contention: No competition for a shared memory bus.
- 8) More complex programming model: Developers must handle data partitioning, communication, and synchronization manually.

7) Explain the difference between shared memory and distributed memory.

shared memory	Distributed memory
1) All processors share a single common memory space	Each processor has its own private memory
2) Communication via shared variables in same address space	Communication via explicit message passing
3) Easier to program (OpenMP, Pthreads)	More complex (requires managing sends/receives)
4) Direct read/write data access to global variables	Data must be copied and sent
5) Requiring using mutexes, semaphores, barriers	Not required in same way, but coordination is needed.
6) Limited scalability	Highly scalability
7) Failure of one core may affect whole system.	Failure of one node is isolated.
8) Interconnect Type: Bus, Crossbar, Ring	Mesh, Torus, Hypercube
9) Example: OpenMP, Pthreads	MPI (Message passing interface).

⑧ Difference Between SIMD systems and MIMD systems.

SIMD (Single Instruction multiple Data)	MIMD (Multiple instruction multiple data)
1) Single instruction stream shared by all processors	Each processor executes its own instruction stream.
2) One control unit for all processing elements	Independent control units per processor
3) Implicit Synchronization	Explicit Synchronization.
4) Data parallelism	Task and data parallelism.
5) Simpler programming model	More complex programming model.
6) Highly effective for regular, repetitive computations	More flexible and efficient for diverse and complex workloads.
7) Limited scalability	Highly scalability
8) Ideal for image processing, simulations, linear algebra	Ideal for web servers, scientific computing, simulations, database
9) Example: Vector processors, GPUs	Example: Multicore CPUs, clusters.

Parallel Computing Module 2.

①

① What is GPU programming in parallel computing.

- 1) GPU programming involves writing code to run on graphic processing units, which are massively parallel processors.
- 2) GPUs follow the SIMD or SIMT (Single instruction multiple thread) model.
- 3) They are highly efficient for data parallel tasks like matrix multiplication, graphics and AI.
- 4) Programming typically involves CUDA or OpenCL.
- 5) Thousands of threads can run concurrently.
- 6) GPU programs must manage device memory vs host memory.
- 7) Data must be explicitly copied between CPU and GPU.
- 8) Performance depends heavily on memory access patterns and thread synchronization.

② What are Hybrid systems and how are they programmed?

- 1) Hybrid systems combine shared memory (OpenMP) and distributed memory (e.g. MPI) programming models.
- 2) A common example is using MPI between nodes and OpenMP of threads within each node.
- 3) They allow exploiting multicore CPUs and cluster together.
- 4) Hybrid programming can reduce communication overhead within a node.
- 5) It increases programming complexity due to mixing models.
- 6) Proper Load balancing is crucial to avoid idle resources.

- *) Used in modern supercomputers. eg: Fugaku
- *) Programmers must be careful to avoid false sharing and memory bandwidth issues

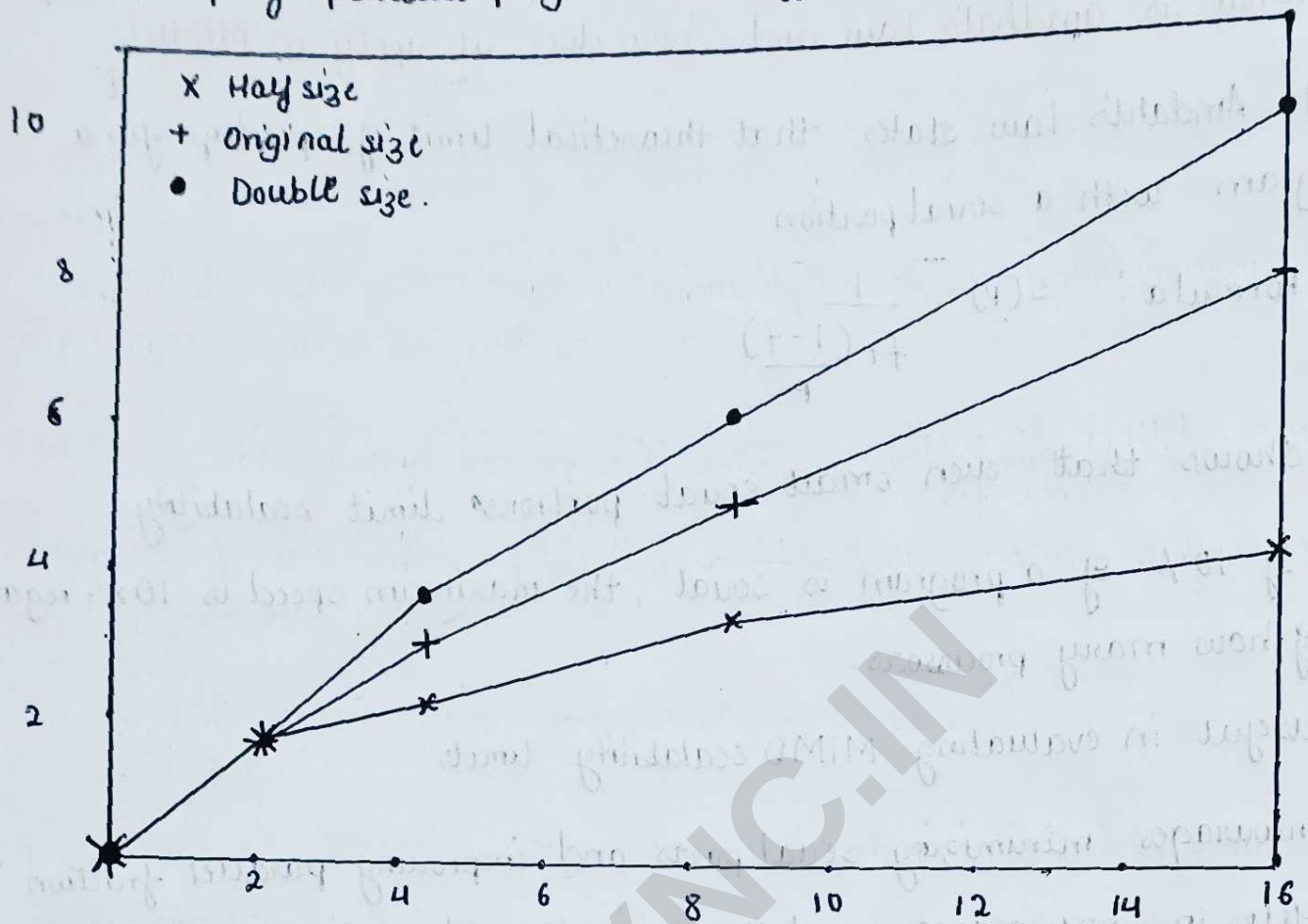
③ Explain the speedup and efficiency in MIMD systems.

- 1) Speedup is the ratio of serial execution time to parallel execution time
- 2) Speedup formula: $\text{speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$
- 3) Efficiency = $\frac{\text{Speedup}}{\text{Number of processors.}}$
- 4) Overheads such as communication, synchronization and idle time reduce speedup.
- 5) Efficiency drops as the number of processors increases (diminishing returns).
- 6) Useful to evaluate performance of MIMD program on clusters or multi-CPU systems.
- 7) Speedup is influenced by task granularity, communication frequency, and load balance.
- 8) Real world application usually target 60-90% efficiency.
- 9) Speedup & efficiency of a parallel program of different problem sizes.

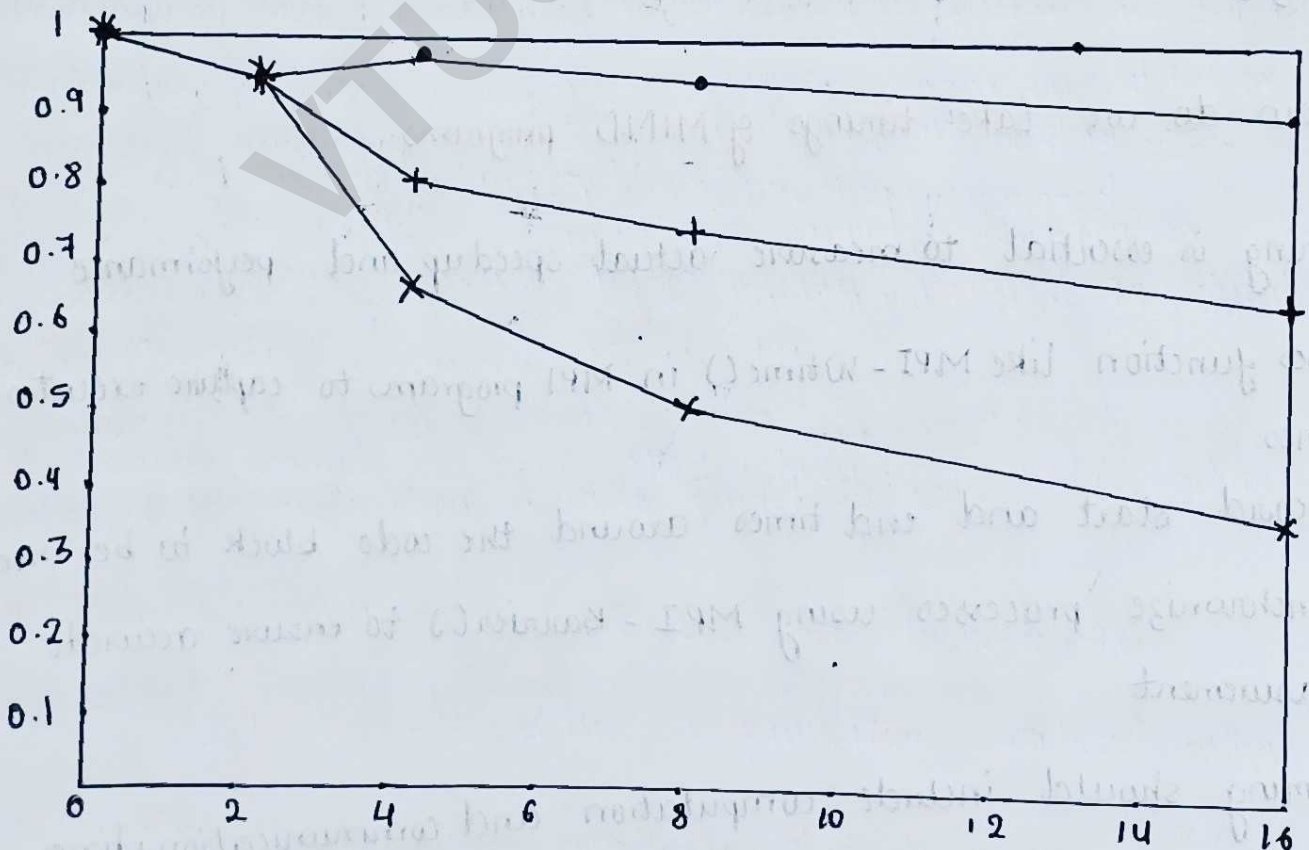
	P	1	2	4	8	16
Hay	S	1.0	1.9	3.1	4.8	6.2
	E	1.0	0.95	0.78	0.60	0.39
Original	S	1.0	1.9	3.6	6.5	10.8
	E	1.0	0.95	0.90	0.81	0.68
Double	S	1.0	1.9	3.9	7.5	14.2
	E	1.0	0.95	0.98	0.94	0.89

Speedup of parallel program on different problem sizes.

(3)



Efficiency of parallel program on different problem sizes.



(4) (11) What is Amdahl's Law and how does it apply in MIMD?

- 1) Amdahl's Law states that theoretical limit of speedup for a program with a serial portion.
- 2) Formula:
$$S(P) = \frac{1}{f + \frac{1-f}{P}}$$
- 3) Shows that even small serial portions limit scalability.
- 4) If 10% of a program is serial, the maximum speed is 10x, regardless of how many processors.
- 5) Useful in evaluating MIMD scalability limits.
- 6) Encourages minimizing serial parts and improving parallel fraction.
- 7) Helps in performance modelling of real world applications.
- 8) Extended by Gustafson's laws for weak scaling (increasing problem size).

(5) How do we take timings of MIMD programs.

- 1) Timing is essential to measure actual speedup and performance.
- 2) Use function like `MPI_Wtime()` in MPI programs to capture execution times.
- 3) Record start and end times around the code block to be measured.
- 4) Synchronize processes using `MPI_Barrier()` to ensure accurate measurement.
- 5) Timing should include computation and communication phases.
- 6) It helps detect bottlenecks and optimize load balancing.
- 7) Multiple runs should be averaged to get stable results.

8) Tools like TAU, gprof, and VTune also help in ~~profile~~ profiling.

⑥ What factors affect GPU performance.

- 1) Memory bandwidth and latency: fastest performance, if data stays in GPU memory
- 2) Occupancy: the number of active threads per multiprocessor.
- 3) Coalesced memory access: sequential threads should access sequential memory
- 4) Branch divergence: reduces performance when threads take different execution paths
- 5) Kernel launch overhead: frequent small kernel launches can degrade performance.
- 6) Global vs shared memory: use shared memory for fast intra-block access
- 7) Proper use of thread blocks and grid dimension is critical
- 8) Optimizing data transfer between host and device using asynchronous memory copy improve performance.

⑦ Explain scalability in MIMD systems?

- 1) scalability in MIMD systems refers to a system ability to efficiently increase performance as a number of processor grows.
- 2) A scalable system will show continued speedup as more processors are added - ideally without significantly increasing inefficiencies or overhead.
- 3) scalability is the capacity of a parallel system to solve larger problems or solve problems faster when given more processing resources.

- 4) Scalability measures how the solution time changes with more processors for a fixed problem size.
- 5) Scalability measures how the solution time changes when problem size increases ⁱⁿ proportionally with the number of processors
- 6) In MIMD, performance drops if processors spend more time in communicating than computing.
- 7) Uneven distribution of work across processors can severely reduce scalability.
- 8) MIMD systems must balance task even when processors follow different instruction paths.
- 9) MIMD system with faster interconnects, lower latency and optimized topology tend to scale better
- 10) A system is scalable if it maintains good efficiency as processor increases.

$$\text{Scalability} = \text{Efficiency} \times \text{Number of processors.}$$

① Explain MPI functions.

→

- 1) Initialization and Finalization: These set up and clean up the environment.
 - a) MPI-Init (&argc, &argv): Initializes the MPI environment
 - b) MPI-Finalize(): Ends the MPI session
 - c) MPI-Abort (comm, errorcode): Terminates MPI program abnormally.
- 2) Process Identification: Used to determine which process is running what
 - a) MPI-Comm-size (MPI-COMM-WORLD, &size): Get the number of processes
 - b) MPI-Comm-rank (MPI-COMM-WORLD, &rank): Get the unique ID of processes
 - c) MPI-Get-processor-name (name, &length): Gets the hostname on which process is running.
- 3) Point to point Communication: Used for sending and receiving messages between two processes.
 - a) MPI-Send (data, count, datatype, destination, tag, comm)
 - b) MPI-Recv (buffer, count, datatype, source, tag, comm, &status)

Eg: A sends a message with tag1; B can choose to only receive message with tag1
- 4) MPI Datatypes: Used to define the kind of data being sent or received
 - a) MPI-INT, MPI-FLOAT, MPI-DOUBLE
 - b) Derived Datatypes: MPI-Type-create-struct
- 5) Collective Communication: These involve all processes in a communicator
 - a) MPI-Bcast(): Broadcast from one to all
 - b) MPI-Scatter(): Distribute chunks from root to all
 - c) MPI-Gather(): Collect chunks from all to root

- d) MPI Allgather(): All collect from all
- e) MPI-Reduce(): Combine all into one
- f) MPI-Allreduce(): Like reduce but result goes to all processes.

6) File I/O in MPI:

- a) MPI-file-open():
- b) MPI-file-read()
- c) MPI-file-write()

7) Derived Datatypes: Useful when communicating structured / non contiguous memory.

- a) MPI-Type-creat-struct()
- b) MPI-Type-commit()

② Define collective communication in MPI? Explain different types of collective communication with example:

→ Collective Communication: Collective Communication in MPI refers to communication operation that involves a group of processes. Collective Communication involves all process in a communicator.

Types of collective Communication:

- 1) Broadcast (MPI-Bcast): Sends data from one process to other processes in the communicator.

Ex: int data

```
if (rank == 0) {
    data = 100;
}
```

```
MPI-Bcast(&data, 1, MPI-INT, 0, MPI-COMM-WORLD);
```


2) Scatter (MPI-Scatter): Distributes chunk of data, from the root process to all processes

Eg: int sendbuf[4] = {1, 2, 3, 4};

int recvbuf;

MPI_Scatter (sendbuf, 1, MPI_INT, &recvbuf, 1, MPI_INT, 0, MPI_COMM_WORLD);

3) Gather (MPI-Gather): Collects data from all process to root process

Eg: int sendbuf = rank;

int recvbuf[4];

MPI_Gather (&sendbuf, 1, MPI_INT, recvbuf, 1, MPI_INT, 0, MPI_COMM_WORLD);

4) Allgather (MPI-Allgather): Like MPI-Gather, but all process receive the gathered data.

Eg: int sendbuf = rank;

int recvbuf[4];

MPI_Allgather (&sendbuf, 1, MPI_INT, recvbuf, 1, MPI_INT, MPI_COMM_WORLD);

5) Reduce (MPI-Reduce): Combines values from all processes using a specified operation and stores the result in the root process

Eg: int value = rank;

int sum;

MPI_Reduce (&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

6) Allreduce (MPI-Allreduce): Like MPI-Reduce, but the result is distributed to all process:

Eg: int value = rank;

int sum;

MPI_Allreduce (&value, &sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

③ Explain the Trapezoidal rule in MPI.

→ Trapezoidal Rule in MPI:

- 1) The Trapezoidal Rule is a numerical integration method used to estimate the definite integral of function $f(x)$ over an interval of $[a, b]$.
- 2) When parallelized using MPI, the work is distributed among multiple processes to speed up the computation.
- 3) Each process can compute a part of the integral independently.
- 4) Ideal for distributed systems where each processor handles a portion of the total interval.
- 5) Formula of trapezoidal Rule: If the interval $[a, b]$ is divided into n equal subintervals.

Width of each subinterval: $h = \frac{b-a}{n}$

Approximate Area: $\text{Integral} \approx h \left[\frac{f(x_0)}{2} + f(x_1) + f(x_2) + \dots + f(x_n) + \frac{f(x_n)}{2} \right]$

6) Parallel steps in MPI:

* Step 1: Master Process Reads Input:

- Input value a, b and n are read by process 0
- These values are broadcasted to all other processes using MPI-Bcast().

* Step 2: Each process computes Local Integral:

- Divide total interval $[a, b]$ into comm_sz parts (number of processes);
- Each processor computes:

$$\text{local_n} = n / \text{comm_sz};$$

$$h = (b-a) / n;$$

$$\text{local_a} = a + \text{my_rank} * \text{local_n} * h;$$

$$\text{local_b} = \text{local_a} + \text{local_n} * h;$$

$$\text{local_integral} = \text{Trap}(\text{local_a}, \text{local_b}, \text{local_n}, h);$$

* Step 3: Combine Results: Use MPI-Reduce () to sum up all local integrals and collect the final result in process 0:

MPI-Reduce (&local-integral, &total-integral, 1, MPI-DOUBLE, MPI-SUM, 0, MPI-COMM-WORLD);

④ Explain MPI I/O functions. with example.

→ 1) MPI I/O provides a standardized and efficient way for parallel processes to perform file input/output in a distributed memory environment

2) Multiple processes need to read/write data simultaneously.

3) Using only one process for all I/O is inefficient for large data.

4) MPI I/O allows all process to perform I/O concurrently in a coordinated way.

5) Serial I/O: Only one process handles all file reading/writing.

6) Parallel I/O: All MPI processes can access the file simultaneously using MPI function.

7) MPI I/O Functions ():

a) MPI-File-open (): Open a file collectively

b) MPI-File-read (): Read data from file

c) MPI-File-write (): Write data to file

d) MPI-File-close (): Close the file

e) MPI-File-seek (): Move the file pointer to a specific position.

Ex: Writing to a file in Parallel

```
#include <mpi.h>
```

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
```

```
    MPI-File fh;
```

```
    int rank, size;
```

```

char buf[20];
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
sprintf(buf, "Hello from %d\n", rank);
MPI_File_open(MPI_COMM_WORLD, "output.txt", MPI_MODE_CREATE |
MPI_MODE_WRONLY, MPI_INFO_NULL, &fh);
MPI_Offset offset = rank * 20;
MPI_File_write_at(&fh, offset, buf, 20, MPI_CHAR, MPI_STATUS_IGNORE);
MPI_File_close(&fh);
MPI_Finalize();
return 0;
}

```

⑤ Explain MPI derived datatypes with an example.

→ 1) In MPI, derived datatypes allow you to describe and send complex data structures efficiently in a single communication call

2) Derived datatypes communicate non contiguous data without manually copying it into a contiguous buffer

3) Bundle multiple variable of different types into one message.

4) Reduce number of MPI-Send / MPI-Recv calls.

5) Parameters of derived datatypes:

a) block-length[]: Number of items of each type

b) types[]: Basic MPI datatypes

c) displacement[]: offsets of each item from the base address

d) MPI-Type-create-struct(): Creates the new type

e) MPI-Type-commit(): Finalize the type for communication

f) MPI-Type-free(): Cleans up the datatype when done

Ex:

```
#include <mpi.h>

void Build-mpi-type (double *a, double *b, int *n, MPI-Datatype *new
type) {
    int block-lengths [3] = {1, 1, 1};
    MPI-Datatype types [3] = {MPI-DOUBLE, MPI-DOUBLE, MPI-INT};
    MPI-AInt displacement [3];
    MPI-AInt a-addr, b-addr, n-addr;
    MPI-Get-address (a, &a-addr);
    MPI-Get-address (b, &b-addr);
    MPI-Get-address (n, &n-addr);
    displacement [0] = 0;
    displacement [1] = b-addr - a-addr;
    displacement [2] = n-addr - a-addr;
    MPI-Type-create-struct (3, block-lengths, displacement, types, new-type);
    MPI-commit (new-type);
}
```

⑥ Explain the performance evaluation of MPI Programs. with an example.

→ 1) Evaluating the performance of an MPI program helps to determine how fast, efficient and scalable your parallel code is compared with serial version.

2) Execution Time: Time taken to execute the parallel portion. Measured using MPI-wtime().

3) Speedup(): How many times faster the parallel program is compared to the serial version

$$S(n,p) = \frac{T_{\text{serial}}(n)}{T_{\text{parallel}}(n,p)}$$

Rarely achieved due to communication overhead and load imbalance.

Efficiency: Utilization of each process

$$E(n, p) = \frac{S(n, p)}{p} = \frac{T_{\text{serial}}(n)}{p \times T_{\text{parallel}}(n, p)}$$

usually less than 1 due to overhead

- 5) Scalability: How well the program maintains efficiency as the problem size and number of processes increase.
- 6) Overhead in MPI programs: $T_{\text{parallel}} = \frac{T_{\text{serial}}}{p} + T_{\text{overhead}}$, where T_{overhead} includes communication, synchronization, and load imbalance.
- 7) MPI provides a built-in function: `MPI_Wtime()`: returns elapsed wall-clock time

Eg: double start, finish;

start = `MPI_Wtime()`;

finish = `MPI_Wtime()`;

`printf("Elapsed time = %.f seconds\n", finish - start);`

8) Explain parallel sorting algorithm with an example

- 1) In MPI, we often work with data distributed across multiple processes
- 2) A parallel sorting algorithm sorts data such that:
 - Each process holds a sorted portion of global dataset
 - All keys in process q are less than or equal to key in process r for all $q < r$
- 3) Parallel version: Odd even Transposition Sort: Uses multiple phases alternating between even and odd indexed comparisons.
- 4) Processes exchange boundary elements with neighboring processes.
- 5) Phases in the algorithm:
 - a) Even phase: Processes with even ranks compare and exchange with rank + 1.
 - b) Odd phase: Processes with odd ranks compare and exchange with rank + 1.

9) Repeat this process for ~~phases~~ 'p' phases (where p = number of processors)

c) Merge functions:

a) Merge-low(): Keeps smaller half of combined array.

b) Merge-high(): Keeps larger half of combined array.

d) Example:

Time	0	1	2	3
start	15, 11, 9, 16	3, 14, 8, 7	4, 6, 12, 10	5, 2, 13, 1
After local sort	9, 11, 15, 16	3, 7, 8, 14	4, 6, 10, 12	1, 2, 5, 13
After phase 0	3, 7, 8, 9	11, 14, 15, 16	1, 2, 4, 5	6, 10, 12, 13
After phase 1	3, 7, 8, 9	1, 2, 4, 5	11, 14, 15, 16	6, 10, 12, 13
After phase 2	1, 2, 3, 4	5, 7, 8, 9	6, 10, 11, 12	13, 14, 15, 16
After phase 3	1, 2, 3, 4	5, 6, 7, 8	9, 10, 11, 12	13, 14, 15, 16

8) Simple Serial sort: Compares adjacent elements and swaps them if out of order.

```
void Bubble ( int a[], int n) {  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n - i - 1; j++) {  
            if (a[j] > a[j+1]) {  
                int temp = a[j];  
                a[j] = a[j+1];  
                a[j+1] = temp;  
            }  
        }  
}
```

Module 4

① Explain openMP pragmas and directives.

- 1) OpenMP directives are used to instruct the compiler to execute code in parallel.
- 2) They begin with `#pragma omp`
- 3) Common directives:
- `#pragma omp parallel` - Create threads
 - `#pragma omp for` - Split loop iterations
 - `#pragma omp sections` - Divide work between threads.
- 4) Directives control thread count, data sharing, scheduling and tasking.
- 5) Ignored by compilers that don't support openMP
- 6) used with clauses like private, shared, reduction, schedule.
- 7) Makes parallelism easy without complex thread code
- 8) Great for multicore CPUs and shared memory systems.

Ex: `#include <stdio.h>`

`#include <stdlib.h>`

`#include <omp.h>`

`void Hello(void);`

`int main (int argc, char* argv[]) {`

`int thread-count = atoi (argv[1], NULL, 10);`

`#pragma omp parallel num.threads (thread-count)`

`Hello();`

`return 0;`

`}
void Hello(void) {`

③

```

int my-rank = omp-get-thread-num();
int thread-count = omp-get-num-threads();
printf("Hello from thread %d of %d\n", my-rank, thread-count);
}

```

* Compiling and running OpenMP programs

```

$ gcc -g -fopenmp -o omp_hello omp_hello.c
$ ./omp_hello4

```

Output: Hello from thread 0 of 4

Hello from thread 1 of 4

Hello from thread 2 of 4

Hello from thread 3 of 4

② Explain the Trapezoidal Rule.

- 1) A numerical method to estimate the area under a curve.
- 2) In parallel, the range is split among threads
 - 3) Each thread computes a partial integral
 - 4) use reduction to combine all partial results.
 - 5) Code uses #pragma omp parallel for reduction (+:sum).
 - 6) If each subinterval has the same length and if we define

$h = (b-a)/n$, $x_i = a + ih$, $i = 0, 1, \dots, n$ then our approximation will be

$$h \left[\frac{f(x_0)}{2} + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + \frac{f(x_n)}{2} \right]$$

Program:

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

```

```

void Trap( double a, double b, int n, double * global result p);
int main (int argc, char * argv[]) {

```

```

double global result = 0.0;
double a, b;
int n;

thread count = strtol (argv [1], NULL, 10);
printf ("Enter a, b, and n \n");
scanf ("%lf %lf %d", &a, &b, &n);
printf ("with n = %d trapezoids, own estimate \n", n);
printf ("of the integral from %lf to %lf = %le \n", a, b, global
        result);
return 0;
}

```

```

void Trap (double a, double b, int n, double * global result, p) {
    double h, x, my-result;
    double local_a, local_b;
    int i;
    local local n = n;
    h = (b-a)/n;
    local n = n / thread count;
    local a = a + my rank * local n * h;
    local b = local a + local n * h;
    myresult = (f(local a) + f(local b)) / 2.0;
    for (i = 1; i <= local n - 1; i++) {
        myresult myresult
        x = local-a + i * h;
        my-result += f(x);
    }
    my-result = my-result * h;
    * global-result-p += my-result;
}

```


Q) Explain the Reduction clause in OpenMP.

- 1) ~~used~~ The Reduction clause is used to safely combine results from multiple threads.
- 2) Syntax: reduction (op: var).
- 3) Each thread gets a private copy of the variable.
- 4) Final result is automatically combined after loop.
- 5) Supports operators: +, *, -, &, | etc
- 6) Avoids race conditions in accumulation
- 7) Essential for parallel numerical computing.
- 8) Example: Sum of array elements:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i, n=100;
    double sum=0.0;
    double a[100];
    for (i=0; i<n; i++)
        a[i] = i*1.0;

    #pragma omp parallel for reduction (+:sum)
    for (i=0; i<n; i++)
        sum += a[i];

    print ("sum = %.f\n", sum);
    return 0;
}
```

④ Explain scheduling and loop carried dependency in openMP

→ Scheduling

1) Determines how loop iterations are assigned to threads

2) Clause: schedule (type, chunk-size)

3) Types:

static: equal blocks, fixed thread

dynamic: thread grab blocks when ready

guided: decreasing chunk sizes.

4) Affects load balancing and performance

5) Useful when loop iteration time varies

6) Can reduce idle time and cache issues

7) Helps optimize performance on real workloads

8) Use profiling to choose best schedule.

Loop carried dependency:

1) Occurs when one loop iteration depends on a previous one.

2) Prevents correct parallel execution.

3) Must be restructured to be parallel safe

4) Detection of dependencies is critical in compiler optimization.

5) Can cause wrong results if parallelized blindly.

6) Techniques like prefix-sum help remove dependency.

⑤ Explain producer and consumers in open mp.

-
- 1) In parallel computing, the producer-consumer model is a classic synchronization problem where:
 - a) A producer thread generates data
 - b) A consumer thread processes or uses the data.
 - 2) They both share a resource and must be synchronized to avoid:
 - a) Reading uninitialized data
 - b) Overwriting data that hasn't been consumed yet.
 - 3) Data race: When both threads access the same memory without coordination
 - 4) Deadlock: If both threads wait forever for each other.
 - 5) Bufferflow or underflow: Producer adds data to a full buffer, or consumer reads from an empty buffer.
 - 6) OpenMP provides synchronization tools to manage this safely:
 - #pragma omp critical
 - omp-lock-t / omp-set-lock() and omp-unset-lock()
 - #pragma omp barrier
 - #pragma omp task and #pragma omp taskwait.

7) Example :

```
#include <stdio.h>
#include <omp.h>
#define size 5
int buffer[size];
int count=0;

void produce (int data){
    buffer [count++] = data;
}
```

```
int consume () {
```

```

    return buffer[--count];
}

int main() {
    #pragma omp parallel num-threads(2)
    {
        if (omp_get_thread_num() == 0) {
            for (int i=0; i<size; i++) {
                #pragma omp critical
                {
                    produce(i);
                    print("Produced: %d\n", i);
                }
            }
        } else {
            for (int i=0; i<size; i++) {
                int data;
                #pragma omp critical
                {
                    if (count > 0) {
                        data = consume();
                        print("Consumed: %d\n", data);
                    }
                }
            }
        }
    }
    return 0;
}

```


⑥ Explain Tasking and Thread safety in openMP.

→ Tasking:

- 1) In openMP, tasking allows you to define independent units of task that can be executed asynchronously by threads.
- 2) Task are created using `#pragma omp task`
- 3) Task are not executed immediately - they're queued and picked up by threads.
- 4) `#pragma omp taskwait` ensures that all child tasks of the current task are complete.
- 5) Helps break free from strict loop-based parallelism.
- 6) Can increase load balancing and parallel efficiency.
- 7) Task can have private and shared data clause.

Ex:

```
int fib(int n) {
    int x, y;
    if (n < 2)
        return n;
    #pragma omp task shared(x)
    x = fib(n-1);
    #pragma omp task shared(y)
    y = fib(n-2);
    #pragma omp taskwait
    return x+y;
}

int main() {
    int result;
    #pragma omp parallel
    { #pragma omp single
        result = fib(10);
    }
    printf("Fibonacci(10) = %d\n", result);
}
```

Thread safety:

- 1) A thread safe function can be used by multiple threads simultaneously without error.
- 2) Use `#pragma omp critical`, `omp-lock-t`, or atomic operations.
- 3) Race conditions occurs if two threads access the same variable without synchronization.
- 4) Thread safety ensures correctness and stability.
- 5) Library function may or may not be thread-safe.
- 6) Reentrant functions are always thread safe.
- 7) Example:

```
#pragma omp parallel
{
    #pragma omp critical
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }
}
```

7) Explain caches, Cache Coherence and false sharing in OpenMP.

→ 1) Cache: Local memory for each processor to speed up access.

2) Cache Coherence: Ensures all threads are consistent data

3) False sharing: Thread modify different variables on the same cache line.

Cache:

1) Caches are small, fast memory units close to each CPU core

2) They stored recently accessed data to reduce memory access time.

3) Each thread in openmp may use its own core and cache

4) Faster access to cached data improves parallel performance.

5) Caches reduce contention for main ~~mem~~ memory (RAM).

Cache Coherence:

- 1) Cache coherence ensures all threads see the latest value of shared variable.
- 2) Required because multiple thread may cache the same memory location.
- 3) If one thread updates a variable, other must get the updated value.
- 4) Maintained using hardware level protocols like MESI.
- 5) Guarantees consistency, but doesn't prevent race conditions.

False sharing:

- 1) Happens when multiple threads modify different variables stored in the same cache line.
- 2) Causes cache lines to be invalidated unnecessarily, reducing performance.
- 3) Even though variables are different, they are shared due to memory layout.
- 4) False sharing leads to cache thrashing and slowdowns as thread count increases.
- 5) Hard to detect without profiling tools.

- ① Explain CUDA Trapezoidal rule 1, ● CUDA Trapezoidal rule 2: Improving Performance and CUDA Trapezoidal rule 3: Blocks with more than one warp.

→ CUDA Trapezoidal rule 1:

- 1) Approximates definite integrals by dividing area into trapezoids.
- 2) Each thread computes the area of one or more trapezoids.
- 3) Kernel computes partial sums and stores in global memory.
- 4) Final result computed by host using reduction of partial areas.
- 5) Parallel version is faster than serial for large n .
- 6) Requires careful indexing of subintervals.

CUDA Trapezoidal Rule II: Improving performance.

- 1) uses shared memory to reduce global memory access latency.
- 2) Threads inside a block store partial sum in shared memory.
- 3) use in-block reduction for faster accumulation.
- 4) Fewer calls to ~~cudaMemcpy~~ cudaMemcpy() between host and device.
- 5) optimizing thread/block size improves occupancy.
- 6) use `--syncthreads()` for thread synchronization in block.

CUDA Trapezoidal Rule III: Blocks with more than one ~~warp~~ warp

- 1) One warp = 32 threads, block can have multiple warps.
- 2) Enables more parallelism per block.
- 3) Need to avoid warp divergence for best performance.
- 4) Can apply warp-level primitives.
- 5) Complex problems benefit from more warps per block.

6) Shared memory must be carefully managed to avoid conflicts. (2)

② Explain the GPU architecture.

- 1) GPU is composed of streaming multiprocessors
- 2) Each SM contains warps (group of 32 threads)
- 3) Cores within SMs operate in SIMT (Single instruction multiple Threads) mode
- 4) GPU memory hierarchy:
- a) Global memory (slow but large)
 - b) Shared memory (faster, SM-local)
 - c) Registers (fastest, per-thread)
- 5) Memory coalescing is important for performance
- 6) Thread divergence reduces performance
- 7) Modern GPUs (like Ampere, Turing) have specialized hardware (tensor cores)
- 8) Thread synchronization is key to correct parallel execution.

③ Explain Heterogeneous Computing.

- 1) uses multiple types of processors in a system: CPU + GPU
- 2) CPU handles logic/control tasks; GPU handles data-parallel compute task
- 3) Data must be transferred between host (CPU) and device (GPU)
- 4) CUDA uses:
- `cudaMemcpy()` for data transfer
 - global -- to define GPU kernels
 - device -- and -- host -- for functions.

- 5) Launch configuration: $\langle\langle\langle$ blocks, threads $\rangle\rangle\rangle$
- 6) Need to manage synchronization between host and device.
- 7) Performance depend on overlap of CPU and GPU workloads.
- 8) Example: video editing, AI, physics engine.

④ Explain GPUs and GPGPU

→ 1) GPU (Graphics processing unit)

- a) Specialized hardware originally designed for rendering graphics.
- b) Contains hundreds to thousands of lightweight cores for parallel tasks.
- c) Uses SIMD/SIMT architecture - execute same instruction on multiple data
- d) Ideal for data parallel operations like vector/matrix calculations.
- e) Found in gaming systems, laptops, servers and supercomputers
- f) Has its own memory hierarchy: global, shared, local and registers.

2) GPGPU (General purpose computing on GPU).

- a) Refers to using the GPU for non-graphics computations.
- b) Enables high-performance computing on scientific and engineering problems.
- c) Leverages GPU parallelism to accelerate general tasks like machine learning, simulations, image processing, cryptography.
- d) GPGPU tasks are written as kernel, launched over many threads.
- f) Offers better performance-per-watt compared to CPUs for parallel tasks.
- g) Requires data transfer between host (CPU) and device (GPU).

(5) Explain the concept of Threads, Blocks and Grids in CUDA

→

1) Threads:

- a) The smallest unit of execution in CUDA
- b) Each thread runs the same kernel function independently.
- c) Each thread has a unique ID to access its portion of data
- d) Accessed using: `threadIdx.x`, `threadIdx.y`, `threadIdx.z`.
- e) Has its own registry and local memory.

2) Blocks:

- a) A group of threads that can cooperate via shared memory.
- b) Threads within a block can synchronize using `--syncthreads()`.
- c) Indexed using: `blockIdx.x`, `blockIdx.y`, `blockIdx.z`.
- d) Block size is defined by: `blockDim.x`, `blockDim.y`, `blockDim.z`.
- e) Max threads per block is typically 1024 (depends on device).

3) Grids:

- a) A collection of blocks launched to execute a kernel.
- b) Enables execution of large scale parallel workloads.
- c) Indexed using: `gridDim.x`, `gridDim.y`, `gridDim.z`.
- d) Organizes how computation is distributed on the device.
- e) Each block in the grid operates independently of others.

(6) What are the key components of Nvidia grid architecture that enable efficient parallel processing?

→ 1) ~~streaming~~ Streaming Multiprocessors

- Core execution • units of NVIDIA GPUs.
- Each SM contains

Cores (CUDA cores) for arithmetic operations

Registers for thread-local storage

Shared memory for intra block data sharing

Instruction schedulers.

- 2) CUDA cores
- Basic processing elements in each SM
 - Execute arithmetic and logical instructions
 - Thousands of CUDA cores in modern GPUs

3) Warps:

- Group of 32 threads that execute instruction in lockstep
- Threads within a warp share the same program counter
- Hardware schedules warp, not individual threads.

4) Thread Blocks:

- Group of threads that:

Execute on the same SM

Share shared memory

Can be synchronized via `--syncthreads()`.

5) Grids:

- Collection of thread blocks
- Blocks in grid can execute independently.
- No synchronization across blocks.

6) Shared memory:

- Fast, user-managed memory within each SM
- Allow data exchange between threads.

7) Global memory:

- Main memory space on the GPU
- Accessible by all threads across all blocks.

8) Registers:

- Fastest memory: private to each thread
- Used for storing thread-specific variables.

Q) Instruction scheduler and Warp scheduler:

- Allocates instruction to warps.
- Handles branch divergence and instruction level parallelism.
- Helps hide memory latency and maintain high-throughput.